TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

A Case Study Report of

Operating System

on

Terminal Multiplexer (shellmux)

Submitted By:
Nitesh Baniya (PUL079BCT051)
Utsav Acharya (PUL079BCT093)
Yojana Thapa (PUL079BCT096)

Submitted To:
Department of Electronics & Computer Engineering

March, 2026

Acknowledgements

We would like to express our sincere gratitude to our subject teachecooperationr, **Assistant Professor Santosh Giri**, for his invaluable guidance, continuous encouragement, and for providing us with the opportunity to work on this Operating System project. His clear explanations of core OS concepts throughout the course gave us the foundation to undertake this implementation.

We are equally thankful to our lab instructor, **Assistant Professor Bikal Adhikari**, for his hands-on support during the laboratory sessions and for helping us debug and validate our implementation in a practical environment.

We are thankful to our seniors and peers for sharing their experiences, guidance, and helpful suggestions, which assisted us in overcoming various challenges during the project. We also acknowledge the support and cooperation of our friends, whose motivation and collaborative discussions contributed positively to the success of this project.

Finally, we express our deepest gratitude to our family members for their constant encouragement, understanding, and moral support, which played a vital role in the completion of this project.

Nitesh Baniya (PUL079BCT051)
Utsav Acharya (PUL079BCT093)
Yojana Thapa (PUL079BCT096)

Abstract

This report presents the design and implementation of **shellmux**, a lightweight terminal multiplexer written in **C++17**. The project demonstrates core operating system concepts including process management via `fork()` and `exec()`, inter-process communication (IPC) through UNIX domain sockets, pseudo-terminal (PTY) handling with `openpty()`, and non-blocking I/O multiplexing using `select()`.

The system follows a client-server architecture that allows users to create and manage multiple named terminal sessions from a simple command-line interface. Each session runs an independent `/bin/bash` process connected via a PTY pair. Key capabilities include session creation, interactive attachment and detachment, session renaming, kill operations, uptime tracking, command counting, and a visual colour indicator when inside a session.

The project prioritises code clarity, modularity, and correctness, making it well suited as both a functional tool and an educational demonstration of Unix/Linux systems programming.

Contents

Acknowledgements **i**

Abstract **ii**

1 Introduction **1**
1.1 Project Motivation . . . 1
1.2 Project Objectives . . . 1

2 System Architecture **1**
2.1 Overall Design . . . 1
2.2 Communication Model . . . 2

3 Core Components **2**
3.1 File Structure . . . 2
3.2 Data Structures . . . 3
3.2.1 Message Structure . . . 3
3.2.2 Session Structure . . . 3
3.3 Server Architecture . . . 4

4 Operating System Concepts Implemented **4**
4.1 Process Management . . . 4
4.1.1 Process Creation and Control . . . 4
4.2 Inter-Process Communication (IPC) . . . 5
4.2.1 UNIX Domain Sockets . . . 5
4.3 Pseudo-Terminal (PTY) Management . . . 6
4.3.1 PTY Pair Creation . . . 6
4.3.2 Terminal State Management . . . 6
4.4 Signal Handling . . . 6

5 Key Features Implementation **7**
5.1 Session Management Commands . . . 7
5.2 Session Colour Theme . . . 7
5.3 Session Metadata Tracking . . . 8

6 Technical Challenges and Solutions **8**
6.1 Concurrency Without Threads . . . 8
6.1.1 Challenge . . . 8
6.1.2 Solution . . . 8
6.2 Terminal Restoration . . . 8
6.2.1 Challenge . . . 8
6.2.2 Solution . . . 8
6.3 Shell Exit Detection . . . 8
6.3.1 Challenge . . . 8
6.3.2 Solution . . . 8

7 Performance Analysis **9**
7.1 Memory Usage . . . 9

7.2 I/O Performance 9
7.3 Scalability 9

8 Testing and Validation **9**
8.1 Functional Testing 9
8.2 Error Handling Validation 10

9 Usage Examples **10**
9.1 Basic Workflow 10
9.2 Sample list Output 11

10 Future Enhancements **11**
10.1 Planned Improvements 11
10.2 Potential Extensions 11

11 Lessons Learned **12**
11.1 Systems Programming Insights 12
11.2 Debugging Challenges 12

12 Conclusion **12**

13 References **13**

A Source Code Structure **13**

B Project Repository **14**

C Compilation and Build **14**

1 Introduction

1.1 Project Motivation

Terminal multiplexers are essential tools in modern Unix/Linux environments. They allow developers and system administrators to manage multiple shell sessions within a single terminal window, keep long-running processes alive after disconnecting, and switch between tasks without opening new windows. Popular tools such as `tmux` and GNU Screen provide these capabilities but are complex pieces of software with thousands of lines of code.

This project implements a simplified but fully functional terminal multiplexer from scratch to explore the operating system mechanisms that underpin such tools. By building shellmux, the team gained hands-on experience with process creation, PTY management, UNIX socket communication, and terminal raw-mode I/O — all fundamental topics in operating systems education.

1.2 Project Objectives

The primary objectives of this project are:

1. Implement a client-server architecture using UNIX domain sockets for IPC.
2. Demonstrate process management by spawning shell processes with `fork()` and `exec()`.
3. Use pseudo-terminals (PTY) to give each session a proper interactive shell.
4. Implement non-blocking I/O multiplexing using `select()`.
5. Provide a clean command-line interface for session management.
6. Add visual feedback (colour theme) to indicate when the user is inside a session.
7. Keep the codebase modular, readable, and suitable for academic viva examination.

2 System Architecture

2.1 Overall Design

shellmux follows a **client-server architecture**:

- **Server Process:** Listens on a UNIX domain socket, manages all sessions, and relays I/O between attached clients and their shell processes.
- **Client Process:** Connects to the server, sends commands (new, attach, list, kill, rename, status), and in attach mode acts as a transparent terminal relay.
- **Session:** Each session holds a forked `/bin/bash` process connected to the server via a PTY master file descriptor.

```
+-------------------------------------------------+
|                    SERVER                       |
|                                                 |
|  Session 0 "work"      Session 1 "logs"         |
|  shell_pid=1234        shell_pid=1235           |
|  PTY master_fd         PTY master_fd            |
|         ^  select() relay loop  v               |
|   UNIX Domain Socket (/tmp/termmgr.sock)        |
+------------------+------------------------------+
                   |  Message{type, session_id,
                   |          data_len} + payload
+------------------v------------------------------+
|                    CLIENT                       |
|   raw terminal mode -> stdin/stdout relay       |
+-------------------------------------------------+
```

Figure 1: High-level architecture of shellmux

2.2 Communication Model

The system uses **UNIX domain sockets** (`AF_UNIX`, `SOCK_STREAM`) for all communication between client and server. This choice provides:

- High-performance local communication with no network overhead.
- Reliable, ordered byte-stream delivery.
- Access control through filesystem permissions on the socket file.
- Full compatibility with standard POSIX I/O operations.

All messages use a fixed-size header followed by a variable-length payload, making parsing simple and reliable without a complex framing protocol.

3 Core Components

3.1 File Structure

```
shellmux/
|-- src/
|   |-- main.cpp        # Entry point and CLI argument parsing
|   |-- server.cpp      # Server: socket accept loop, session dispatch
|   |-- server.h        # Server class and Message protocol definition
|   |-- client.cpp      # Client: connect, commands, attach loop
|   |-- client.h        # Client class declaration
|   |-- session.cpp     # Session: fork/exec bash with PTY
|   `-- session.h       # Session class declaration
|-- build/              # Compiled output (git-ignored)
```

```
|-- Makefile
|-- .gitignore
`-- README.md
```

Listing 1: Project file organisation

3.2 Data Structures

3.2.1 Message Structure

Every message exchanged between client and server uses a fixed-size header defined as follows:

```
struct Message {
    enum Type : uint8_t {
        NEW_SESSION  = 1,   // client -> server: create session
        ATTACH       = 2,   // client -> server: attach to session
        LIST         = 3,   // client -> server: list sessions
        DATA         = 4,   // bidirectional: raw I/O bytes
        RESPONSE     = 5,   // server -> client: text reply
        DETACH       = 6,   // client -> server: detach
        ERROR_MSG    = 7,   // server -> client: error
        KILL_SESSION = 8,   // client -> server: kill session
        RENAME       = 9,   // client -> server: rename session
        STATUS       = 10,  // client -> server: server stats
    };
    Type type;
    int  session_id;   // target session ID
    int  data_len;     // bytes of payload following this header
};
```

Listing 2: Wire protocol message header (server.h)

After the header, `data_len` raw bytes of payload are sent. The fixed-size header makes parsing straightforward: read exactly `sizeof(Message)` bytes first, then read `data_len` bytes for the payload.

3.2.2 Session Structure

```
class Session {
public:
    int         id;            // unique session ID
    std::string name;          // human-readable name (renameable)
    pid_t       shell_pid;     // PID of the forked bash process
    int         master_fd;     // PTY master file descriptor
```

```
    bool        active;       // is the session alive?
    time_t      created_at;   // creation timestamp
    int         cmd_count;    // approximate command count

    bool start();             // fork + exec /bin/bash
    void stop();              // SIGKILL + waitpid + close
       fds
    bool write_to_shell(const char* buf, int len);
    int  read_from_shell(char* buf, int max_len);
    std::string uptime_str() const;  // e.g. "3m 20s"
};
```

Listing 3: Session class (session.h)

3.3 Server Architecture

The server uses an **event-driven design** with `select()` to handle I/O from multiple file descriptors without threading:

```
void Server::relay_loop(int client_fd, Session* sess) {
    char buf[MAX_PAYLOAD];
    while (sess->active) {
        fd_set rfds;
        FD_ZERO(&rfds);
        FD_SET(client_fd,       &rfds);
        FD_SET(sess->master_fd, &rfds);
        int maxfd = std::max(client_fd, sess->master_fd) + 1;

        timeval tv{ 0, 100000 };  // 100ms timeout
        select(maxfd, &rfds, nullptr, nullptr, &tv);

        // client -> shell: forward keystrokes
        if (FD_ISSET(client_fd, &rfds)) { /* ... */ }

        // shell -> client: forward output
        if (FD_ISSET(sess->master_fd, &rfds)) { /* ... */ }
    }
}
```

Listing 4: Relay loop using select() (server.cpp)

4 Operating System Concepts Implemented

4.1 Process Management

4.1.1 Process Creation and Control

Each session spawns an independent `/bin/bash` process using the classic **fork-exec pattern**:

```
shell_pid = fork();

```

```
if (shell_pid == 0) {
    // Child process
    close(master_fd);       // child does not need the master end

    setsid();               // create new session; child becomes leader
    ioctl(slave_fd, TIOCSCTTY, 0); // set slave PTY as controlling terminal

    // Redirect stdin/stdout/stderr to the PTY slave
    dup2(slave_fd, STDIN_FILENO);
    dup2(slave_fd, STDOUT_FILENO);
    dup2(slave_fd, STDERR_FILENO);

    execlp("/bin/bash", "bash", nullptr);
    _exit(1);  // only reached if exec fails
}
// Parent continues with master_fd open
```

Listing 5: fork() + exec() in Session::start() (session.cpp)

Key OS concepts demonstrated here:

- `fork()` — duplicates the process; child gets PID 0 in return value.
- `setsid()` — creates a new process session so the child can have its own controlling terminal.
- `ioctl(TIOCSCTTY)` — assigns the PTY slave as the controlling terminal.
- `dup2()` — redirects file descriptors so the shell reads/writes from the PTY.
- `execlp()` — replaces the child's memory image with `/bin/bash`.
- `waitpid()` — used in `Session::stop()` to reap the child and prevent zombies.

4.2 Inter-Process Communication (IPC)

4.2.1 UNIX Domain Sockets

shellmux uses **UNIX domain sockets** as its IPC mechanism. The server creates a listening socket bound to `/tmp/termmgr.sock`:

```
server_fd_ = socket(AF_UNIX, SOCK_STREAM, 0);

sockaddr_un addr{};
addr.sun_family = AF_UNIX;
strncpy(addr.sun_path, SOCKET_PATH, sizeof(addr.sun_path) - 1);

bind(server_fd_, (sockaddr*)&addr, sizeof(addr));
listen(server_fd_, 5);
```

Listing 6: Server socket setup (server.cpp)

The client connects with a symmetric `connect()` call. All subsequent communication uses a simple header + payload protocol described in Section 3.2.1.

4.3 Pseudo-Terminal (PTY) Management

4.3.1 PTY Pair Creation

A **pseudo-terminal** (PTY) consists of a master/slave pair. The shell writes its output to the slave; the server reads it from the master. This makes the shell believe it is connected to a real terminal, enabling interactive features such as line editing, colours, and job control.

```
int master_fd, slave_fd;
openpty(&master_fd, &slave_fd, nullptr, nullptr, nullptr);

// Make master non-blocking so reads don't hang
int flags = fcntl(master_fd, F_GETFL, 0);
fcntl(master_fd, F_SETFL, flags | O_NONBLOCK);
```

Listing 7: PTY creation with openpty() (session.cpp)

4.3.2 Terminal State Management

The client puts its own terminal into **raw mode** before entering the attach loop, so every keypress is forwarded immediately rather than waiting for the Enter key:

```
void Client::set_raw_mode() {
    tcgetattr(STDIN_FILENO, &g_saved_termios);
    termios raw = g_saved_termios;

    // Disable canonical mode (line buffering) and echo
    raw.c_lflag &= ~(ICANON | ECHO | ISIG);
    raw.c_iflag &= ~(IXON | ICRNL);
    raw.c_cc[VMIN] = 1;  raw.c_cc[VTIME] = 0;

    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}
```

Listing 8: Raw terminal mode (client.cpp)

The original terminal settings are saved and fully restored when the user detaches.

4.4 Signal Handling

Session termination uses **POSIX signals**:

```
void Session::stop() {
    if (!active) return;
    active = false;
```

```
    kill(shell_pid, SIGTERM);    // polite request to terminate
    usleep(100000);              // wait 100ms
    kill(shell_pid, SIGKILL);    // force kill if still running
    waitpid(shell_pid, nullptr, 0);  // reap zombie
    close(master_fd);
}
```

Listing 9: Session teardown with signals (session.cpp)

5 Key Features Implementation

5.1 Session Management Commands

Table 1 summarises all available commands:

Command	Action	OS Concept
`./build/app server`	Start the server	Socket bind/listen
`./build/app new <name>`	Create a named session	fork + exec + PTY
`./build/app list`	List sessions with status	IPC / UNIX socket
`./build/app attach <id>`	Attach interactively	select() relay
`./build/app kill <id>`	Kill a session	SIGTERM/SIGKILL
`./build/app rename <id> <n>`	Rename a session	IPC message
`./build/app status`	Server uptime & stats	IPC message
`./build/app help`	Print usage guide	—

Table 1: shellmux command reference

5.2 Session Colour Theme

When a client attaches to a session, the terminal background is changed to dark navy blue and the foreground to bright white using ANSI 256-colour escape codes. This provides immediate visual feedback that the user is inside a managed session:

```
// On ATTACH -- dark blue background, bright white text, clear screen
const char* SESSION_THEME =
    "\033[48;5;17m"    // background: dark navy (colour #17)
    "\033[38;5;15m"    // foreground: bright white (colour #15)
    "\033[2J\033[H";   // clear screen, cursor to top-left
write(STDOUT_FILENO, SESSION_THEME, 18);

// On DETACH -- reset all attributes and clear screen
write(STDOUT_FILENO, "\033[0m\033[2J\033[H", 11);
```

Listing 10: Visual colour theme on attach/detach (client.cpp)

5.3 Session Metadata Tracking

Each session records:

- `created_at` — Unix timestamp of session creation, used to compute and display uptime in `list` output.
- `cmd_count` — incremented each time a newline or carriage return is forwarded to the shell, giving an approximate count of commands run in the session.

6 Technical Challenges and Solutions

6.1 Concurrency Without Threads

6.1.1 Challenge

The server must simultaneously watch the client socket (for incoming keystrokes) and the PTY master (for shell output) without blocking on either.

6.1.2 Solution

The `select()` system call multiplexes both file descriptors in a single thread. A 100 ms timeout keeps the loop responsive and allows the server to check whether the session is still active. This avoids the complexity and race-condition risks of multi-threading.

6.2 Terminal Restoration

6.2.1 Challenge

If the client process exits abnormally while in raw mode, the user's terminal is left in a broken state where typed characters do not appear.

6.2.2 Solution

The original `termios` settings are saved at the start of `set_raw_mode()` and restored in `restore_terminal()`, which is called both on clean detach and in the `Client` destructor. This guarantees terminal cleanup even in error paths.

6.3 Shell Exit Detection

6.3.1 Challenge

When the user types `exit` inside a session, the shell process terminates. The PTY master returns `EIO` on further reads, which must be detected and handled gracefully.

6.3.2 Solution

In `relay_loop()`, a return value less than zero from `read_from_shell()` is treated as a shell exit event. The server marks the session inactive, sends a notification message to the client, and breaks out of the relay loop cleanly.

7 Performance Analysis

7.1 Memory Usage

shellmux uses fixed-size stack and heap allocations for predictable memory consumption:

Component	**Per Session**	**10 Sessions**
Session object (metadata)	∼1 KB	∼10 KB
PTY master file descriptor	1 fd	10 fds
I/O relay buffer	4 KB	40 KB
Total	∼5 KB	∼50 KB

Table 2: Approximate memory usage

The deliberate omission of a screen buffer and scrollback history keeps memory usage very low compared to full-featured multiplexers.

7.2 I/O Performance

The event-driven architecture provides efficient I/O handling:

- Single-threaded design eliminates context-switching overhead.
- `select()` with a 100 ms timeout minimises CPU usage during idle periods.
- UNIX domain sockets provide lower latency than TCP loopback.
- Non-blocking reads on the PTY master prevent the relay loop from stalling.

7.3 Scalability

Current design limits:

- Maximum 10 concurrent sessions (configurable via `MAX_SESSIONS` in `server.h`).
- One client attached at a time (by design — simplifies the server).
- The session list uses linear search, suitable for the small scale of this project.

8 Testing and Validation

8.1 Functional Testing

The following scenarios were tested and verified:

1. **Session lifecycle** — create, attach, run commands, detach, list, kill.
2. **Multiple sessions** — creating several sessions and verifying all appear in `list` with correct uptime.
3. **Rename** — renaming a live session and confirming the new name appears in subsequent `list` calls.

4. **Session limit** — attempting to create an 11th session returns an error.
5. **Attach to dead session** — server returns a descriptive error message.
6. **Shell exit** — typing `exit` inside a session gracefully detaches the client and marks the session dead.
7. **Colour theme** — terminal visually changes to dark blue on attach and resets on detach.
8. **Command counter** — `cmd_count` increments with each Enter keypress and is visible in `list`.

8.2 Error Handling Validation

Tested error conditions include:

- Running a client command before the server is started — clear error message is printed.
- Attaching to a non-existent session ID — server returns `ERROR_MSG`.
- Server process killed — client detects broken socket and exits cleanly.
- Invalid command argument — usage hint is printed.

9 Usage Examples

9.1 Basic Workflow

Open two terminal windows side by side.

Terminal 1 — Start the server (keep this running):

```
./build/app server
```

Expected output:

```
+=====================================+
|  Terminal Manager Server -- RUNNING  |
+=====================================+
  Socket : /tmp/termmgr.sock
  Limit  : 10 sessions
```

Terminal 2 — Use client commands:

```
# Create two sessions
./build/app new work
./build/app new logs

# List all sessions
./build/app list

# Attach to session 0 (terminal turns dark blue)
./build/app attach 0
```

```
# --- now inside session 0 ---
ls
whoami
# Press Ctrl+D to detach
# --- back to normal terminal ---

# Rename session 1
./build/app rename 1 monitoring

# Check server status
./build/app status

# Kill session 1
./build/app kill 1

# Confirm it is gone
./build/app list
```

Listing 11: Complete example session

9.2 Sample list Output

```
 ID   NAME              STATUS     UPTIME     CMDS
---------------------------------------------
   0  work               active     2m 15s     7
   1  monitoring         active     1m 3s      0
```

10 Future Enhancements

10.1 Planned Improvements

1. **SIGCHLD handler** — automatically detect when a shell exits using `signal(SIGCHLD, ...)` instead of polling, making session cleanup more immediate.
2. **Window resize forwarding** — forward the client's terminal dimensions to the PTY using `ioctl(TIOCSWINSZ)`, fixing layout issues in full-screen programs like `vim` and `nano`.
3. **Session logging** — write all shell output to a log file at `~/.shellmux/session-<id>.log` for later review.
4. **Configuration file** — read socket path and session limit from `~/.shellmuxrc`.

10.2 Potential Extensions

- Session persistence across server restarts using `shm_open()`.
- Multi-client per session using shared memory and semaphores.
- Scrollback buffer implemented as a ring buffer.
- Mouse support via ANSI mouse escape sequences.

- Session recording and playback (similar to `script(1)`).

11 Lessons Learned

11.1 Systems Programming Insights

This project provided valuable hands-on experience in several areas:

1. **Process Management:** The fork-exec-waitpid pattern and the importance of reaping children to prevent zombie processes.
2. **Terminal Programming:** The difference between a plain pipe and a PTY, and why shells require a real terminal to behave interactively.
3. **IPC Design:** Designing a simple but extensible binary protocol with a fixed-size header eliminates framing ambiguity without adding complexity.
4. **Event-Driven I/O:** Using `select()` to multiplex multiple file descriptors in a single thread is a powerful pattern that avoids threading complexity entirely.
5. **Terminal Modes:** The `termios` API and the importance of saving and restoring terminal settings so the user's shell is not left in a broken state.

11.2 Debugging Challenges

Key debugging experiences included:

- **PTY not behaving as a terminal:** Forgetting to call `setsid()` and `ioctl(TIOCSCTTY)` caused bash to run in non-interactive mode. Fixing the session/process group setup resolved this.
- **Blocking reads on master_fd:** Without `O_NONBLOCK`, `read()` on the PTY master would block indefinitely when the shell had no output. Setting the non-blocking flag fixed the relay loop stall.
- **Terminal left in raw mode:** An early version did not restore `termios` on abnormal exit, leaving the terminal in raw mode. Moving restoration into the destructor solved this permanently.

12 Conclusion

shellmux successfully implements a functional terminal multiplexer in under 700 lines of C++17, demonstrating that even complex-seeming tools are built from a small set of well-understood OS primitives.

The project concretely demonstrates:

- Process creation and control with `fork()`, `exec()`, `waitpid()`, and POSIX signals.
- Pseudo-terminal management with `openpty()`, `setsid()`, and `TIOCSCTTY`.

- Inter-process communication via UNIX domain sockets with a custom binary protocol.
- Non-blocking multiplexed I/O using `select()`.
- Terminal raw mode control with the `termios` API.

The clean, modular codebase (five files, four classes) makes every concept easy to locate and explain. shellmux serves as both a working developer tool and a clear educational demonstration of operating system concepts covered in the Operating System course at Pulchowk Campus.

13 References

1. Stevens, W. Richard, and Stephen A. Rago. *Advanced Programming in the UNIX Environment*, 3rd ed. Addison-Wesley, 2013.
2. Kerrisk, Michael. *The Linux Programming Interface*. No Starch Press, 2010.
3. Rochkind, Marc J. *Advanced Unix Programming*, 2nd ed. Addison-Wesley, 2004.
4. Linux Manual Pages. https://man7.org/linux/man-pages/ — `fork(2)`, `exec(3)`, `openpty(3)`, `select(2)`, `termios(3)`, `unix(7)`.
5. tmux source code and documentation. https://github.com/tmux/tmux/wiki
6. POSIX.1-2017 Standard. IEEE and The Open Group, 2017.

A Source Code Structure

```
shellmux/
|-- src/
|   |-- main.cpp        ~50  lines  Entry point, CLI dispatch
|   |-- server.h        ~60  lines  Message protocol, Server
   class
|   |-- server.cpp      ~220 lines  Server implementation
|   |-- client.h        ~30  lines  Client class declaration
|   |-- client.cpp      ~200 lines  Client implementation
|   |-- session.h       ~30  lines  Session class declaration
|   `-- session.cpp     ~90  lines  Session implementation
|-- build/                          Git-ignored compiled
   output
|-- Makefile
|-- .gitignore
`-- README.md
```

Listing 12: Complete project layout

Total: $\approx$ 680 lines of C++17.

B Project Repository

The complete source code is available at:

https://github.com/niteshbaniya/shellmux

C Compilation and Build

```
# Clone
git clone https://github.com/niteshbaniya/shellmux.git
cd shellmux

# Build (requires g++ with C++17 and libutil)
make

# Clean
make clean

# Run
./build/app server &    # Terminal 1
./build/app help        # Terminal 2
```